Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Машина состояний

otus.ru

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в TG
OTUS Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План полета

- Общие сведения
- Теория конечных автоматов
- Примеры применения
- Реализация в коде
- Рефлексия

Цели вебинара

После занятия вы сможете

1. Знать общие принципы работы машины состояний
2. Применять их при проектировании логики приложений
3. Реализовывать в своих программах

Смысл

Зачем вам это уметь

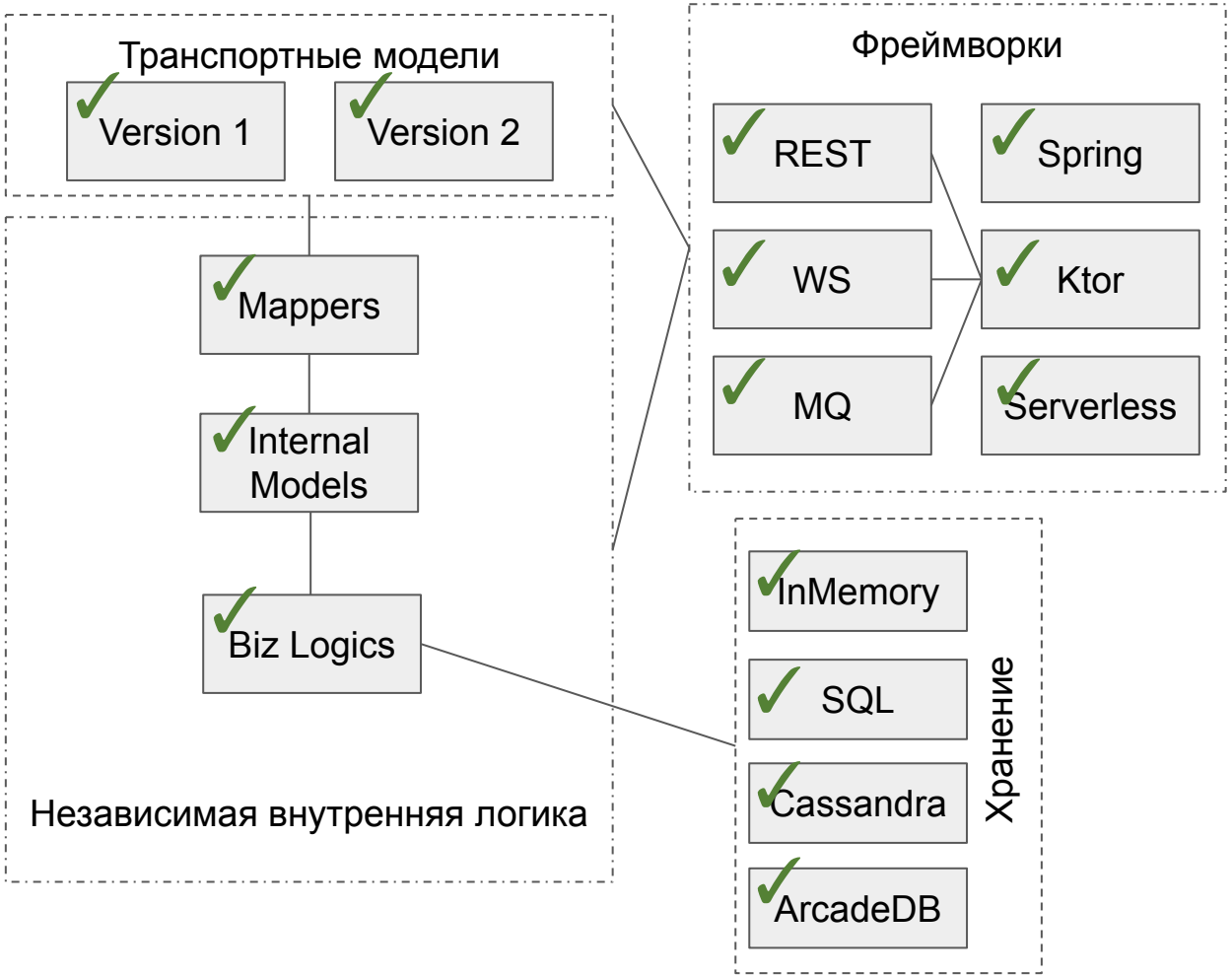
1. Машина состояний является аналогом машинного обучения, но проще, понятнее, более контролируема и быстрее
2. Машина состояний сильно упрощает код и сокращает количество if-ов
3. Работу по настройке машины состояний могут выполнять аналитики

Обзор пройденного

Пройденный материал

Что сделано:

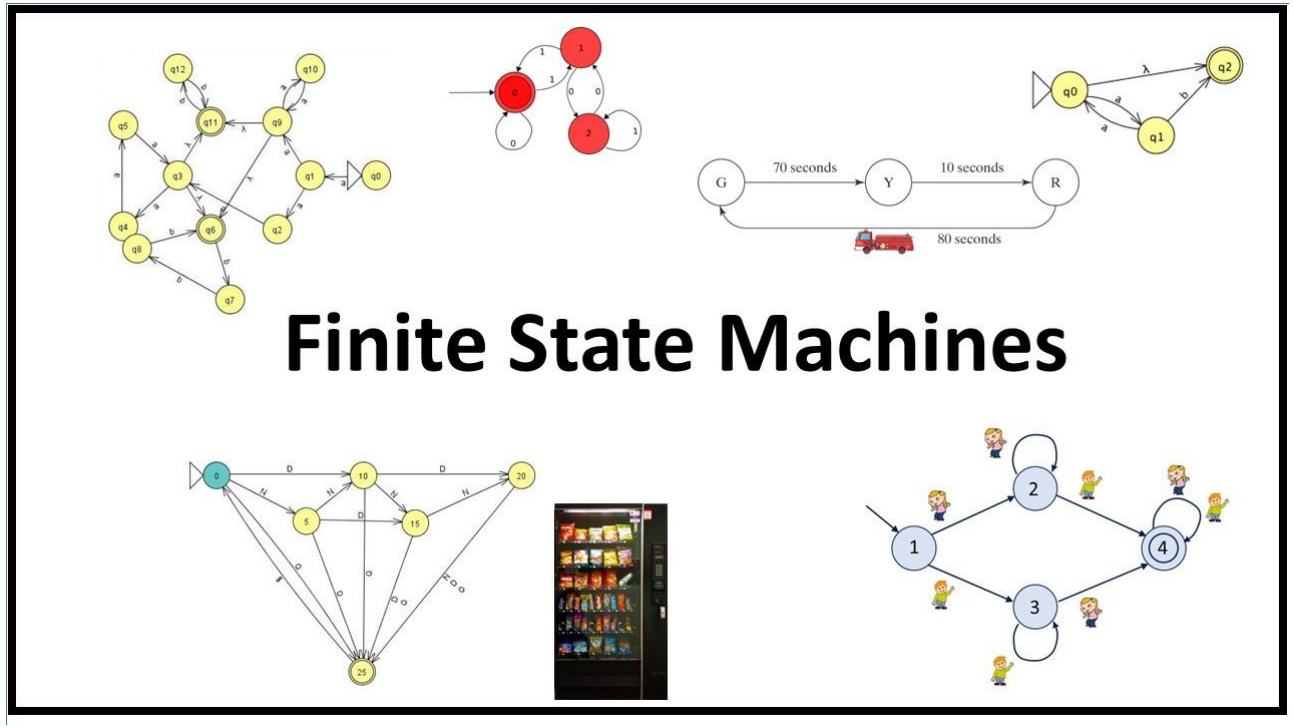
1. Подготовка к разработке:
   - Маркетинг и аналитика
   - Безопасность
   - DevOps
   - Архитектура
   - Тестирование
   - Мониторинг
2. API и транспортные модели
3. Внутренние модели и маперы
4. Фреймворки и транспорт: Spring, Ktor, WS, Rabbit, Kafka
5. Бизнес-логика: библиотека, валидация
6. Базы данных: Репозиторий, конкуренция, InMemory, PG, Cassandra, Gremlin



Общие сведения

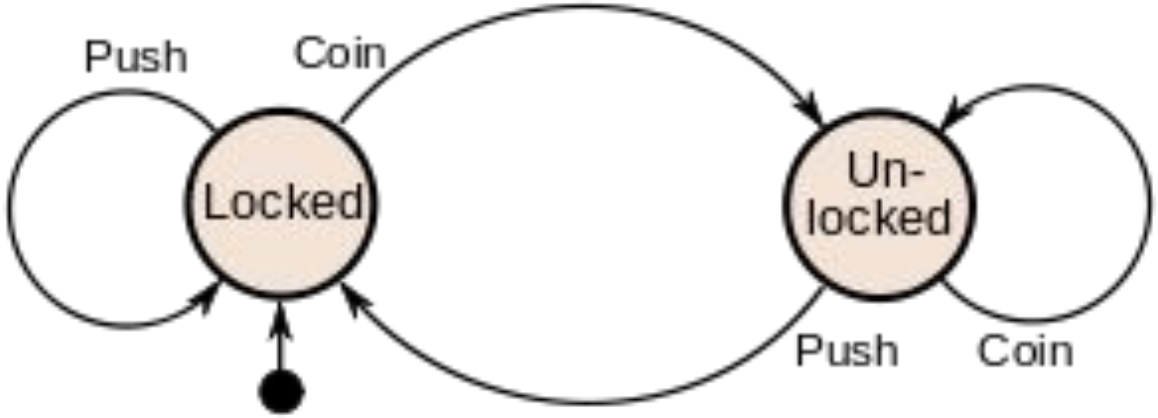
Вопрос?

- Вы пользовались ранее машинами состояний?
- Как вы их использовали?



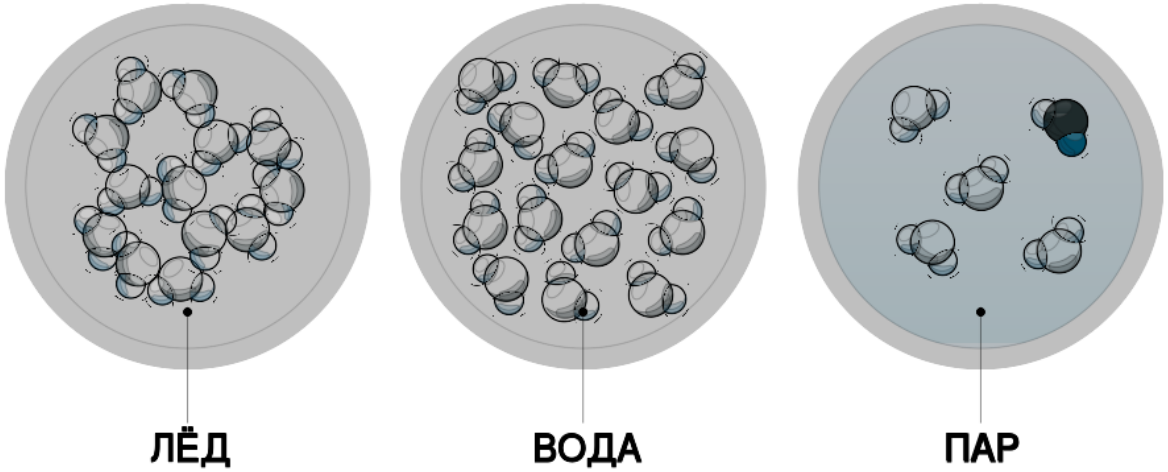
О машине состояний

Коне́чный автомат (КА) в теории алгоритмов — математическая абстракция, модель дискретного устройства, имеющего один вход, один выход и в каждый момент времени находящегося в одном состоянии из множества возможных. Является частным случаем абстрактного дискретного автомата, число возможных внутренних состояний которого *конечно*.



Состояния

Состояние — понятие, обозначающее множество устойчивых значений переменных параметров объекта.



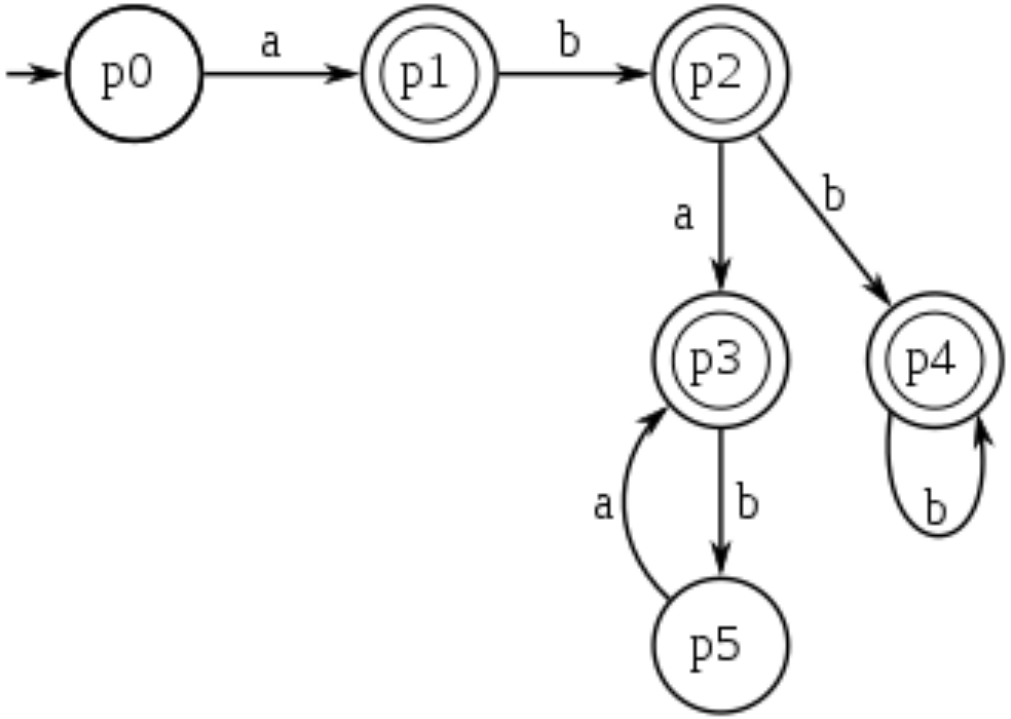
- положения тела человека: лежит, сидит, стоит, ходьба, бег;
- состояние головного мозга: сон, бодрствование;
- агрегатное состояние физического вещества: твёрдое тело (кристаллическое, стеклообразное, жёсткое, гибкое), жидкость (вязкая, жидкая), газ, плазма;
- совокупность материального имущества человека также называется его состоянием (см. богатство, капитал);
- в цифровой электронике: двоичный триггер имеет два устойчивых состояния, троичный триггер имеет три устойчивых состояния и т. д.

Состояние в разработке

- Различные виды действия
  - “Открыто”/”Закрыто – подавать/не подавать напряжение на замок
  - “Ожидание”/”Обработка” – ничего не делать/выполнять бизнес-логику
  - “Tab1”/”Tab2” – отображать в интерфейсе нужную вкладку
- Различные виды реакций
  - “Забанен”/”Активный” – запрет/разрешение оставлять сообщения
- Промежуточные состояния

Переходы между состояниями

- Детерминированные переходы
  - Всегда по заложенному пути
  - используется чаще всего
- Недетерминированные переходы (в играх)
  - На один сигнал в разные состояния
  - Самопроизвольно в случайный момент времени



Сигналы

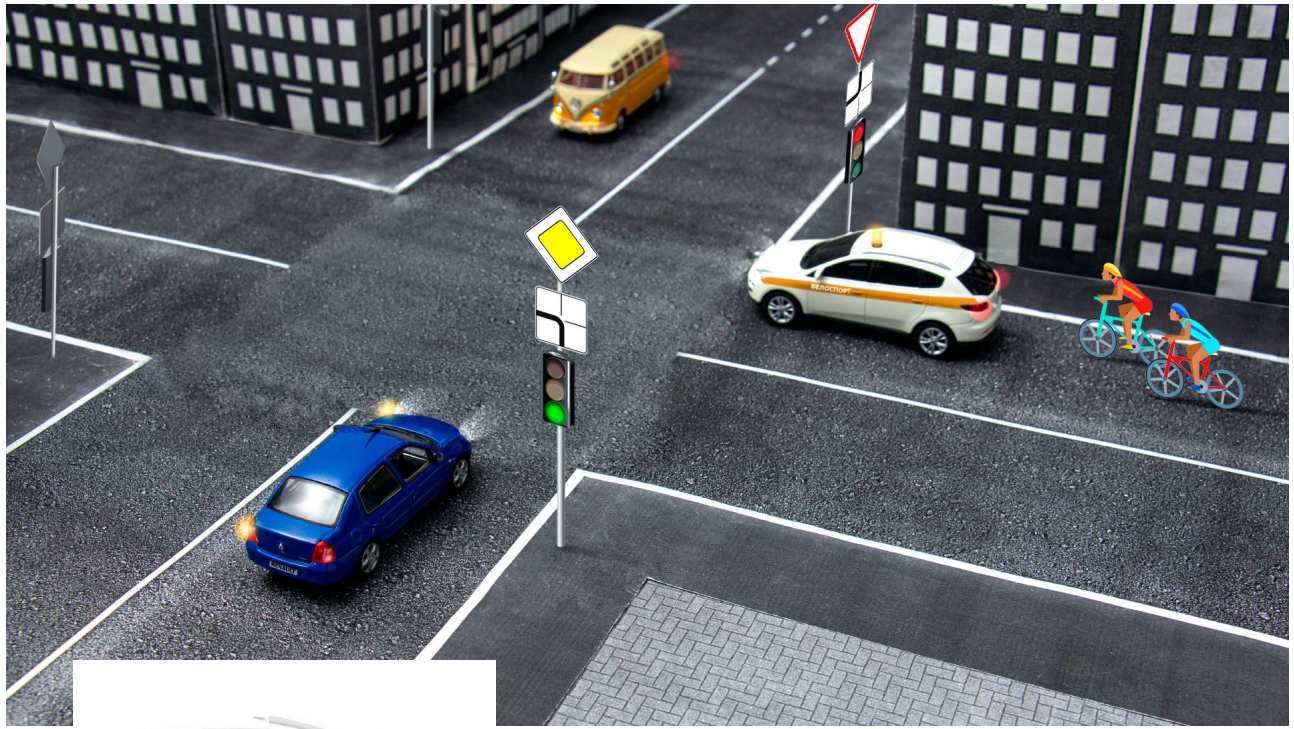
Служат для инициирования перехода

В зависимости от характеристик сигнала, переход в разные состояния

Импульсные сигналы (получили сигнал)

Непрерывные сигналы (с герконового датчика)

Сигналы многокомпонентные складываются из серии других сигналов



Машина состояний

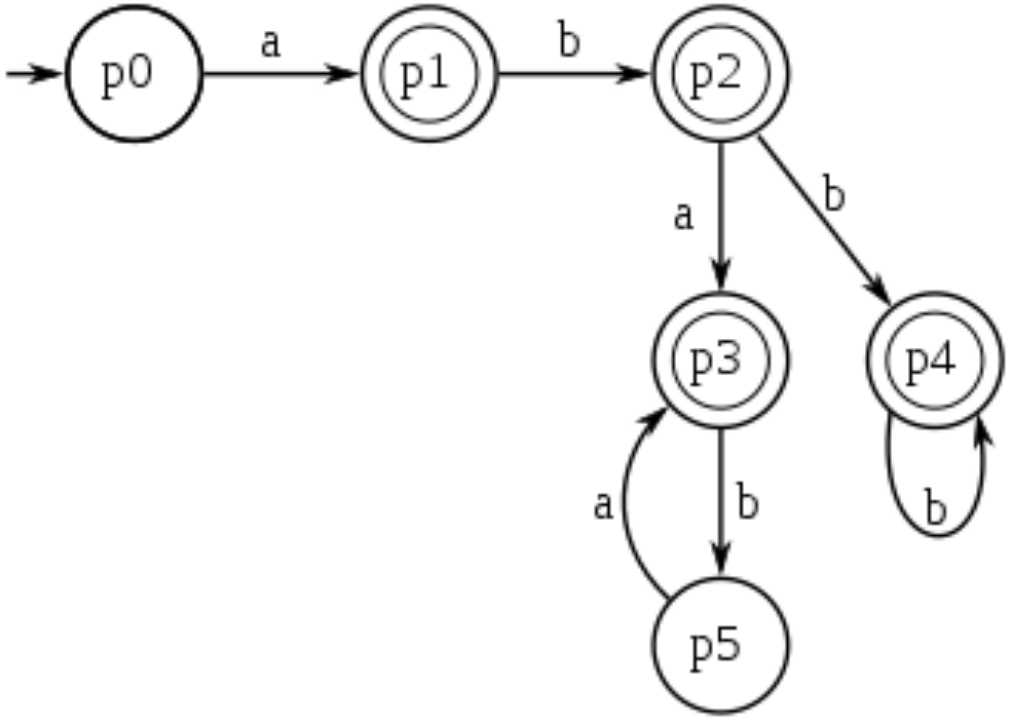
Конечный набор состояний

Переходы между состояниями

Под действием конечного числа типов сигналов

Ациклическая машина состояний (характеризует процесс с завершением)

Циклическая машина состояний



Приведите примеры машин состояний

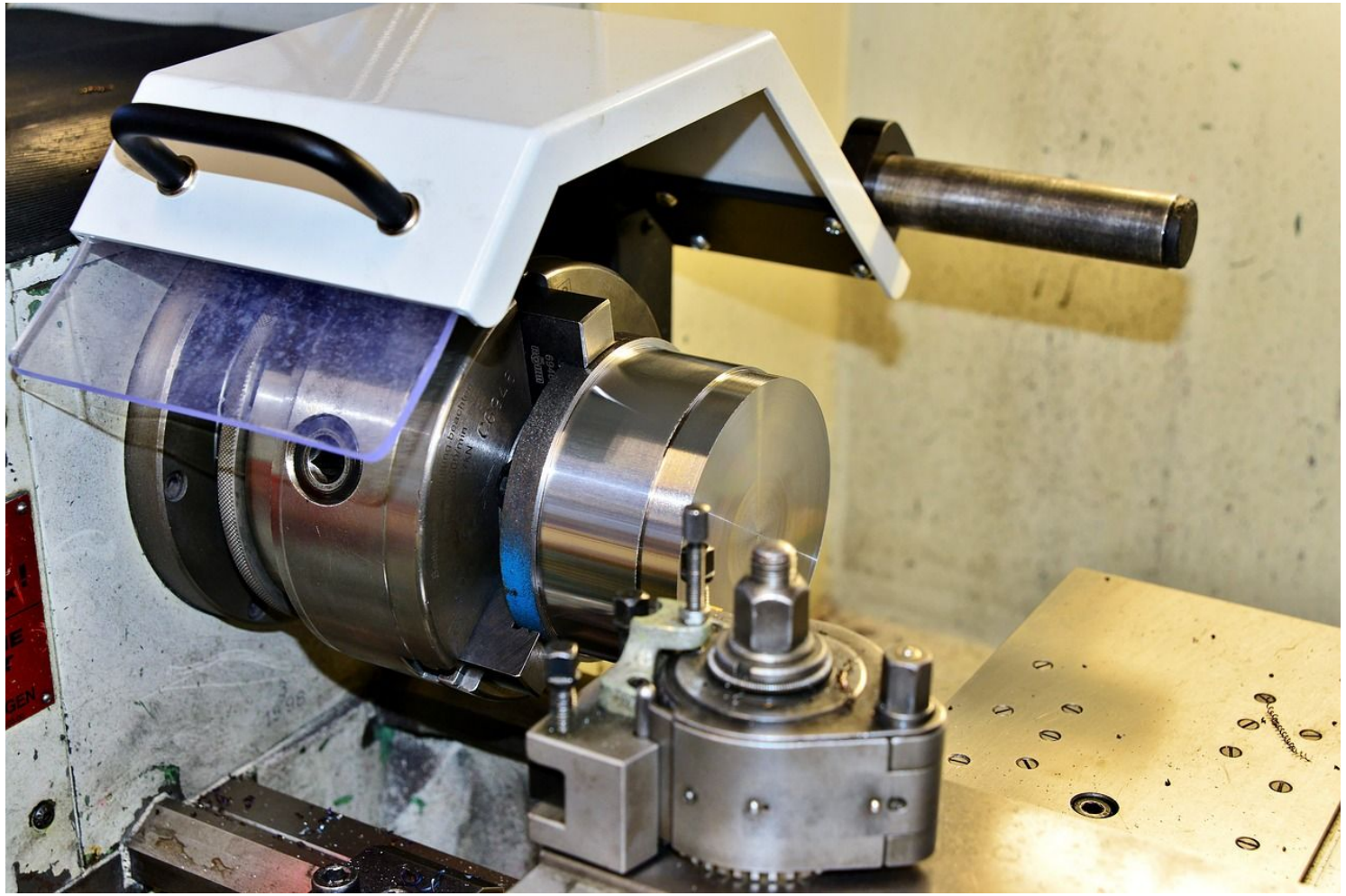
Какие можете придумать машины состояний?

Представление машины состояний

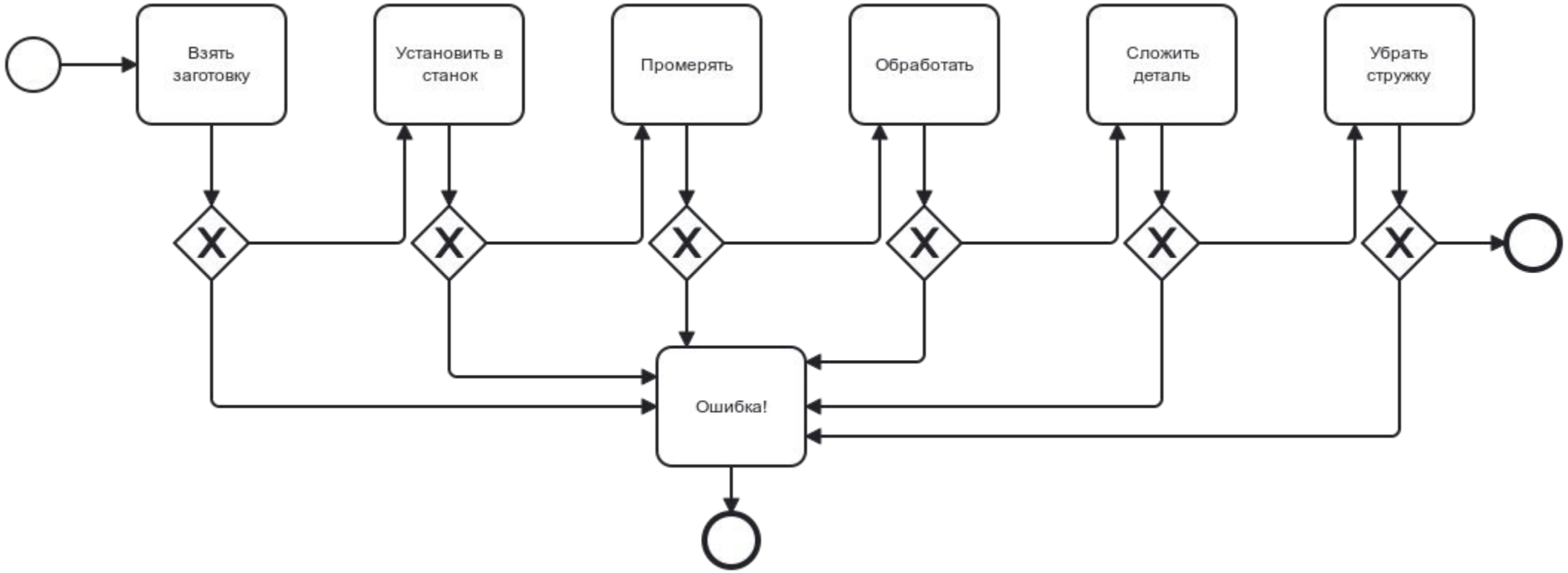
Пример: обработка детали станком

Операции:

- Взять заготовку
- Установить в станок
- Промерять
- Обработать
- Сложить деталь
- Убрать стружку



Версия аналитика



Глупое решение - if-else
Правильное решение - Машина состояний

Состояния

Enum

Sealed class

разбивка помогает качественнее декомпозировать состояния

Состояния (версия аналитика):
- StandBy
- TakeWorkpiece
- MoveWorkpiece
- MeasureWorkpiece
- Handle
- PlaceDetail
- CleanMachine

Состояния:
- **StandBy**
- **Starting**
- **Processing**
- **Finishing**
- **Failure**

3 Состояния Finishing:
- StandBy
- PlaceDetail
- CleanMachine
- MakeReport
- Finishing
- Failure

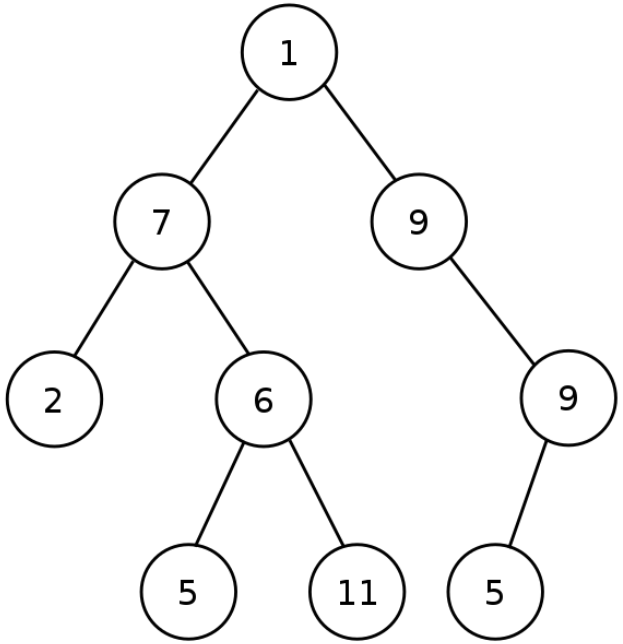
2 Состояния Processing:
- StandBy
- CutterPositioning
- Cut
- FinishCutting
- Failure

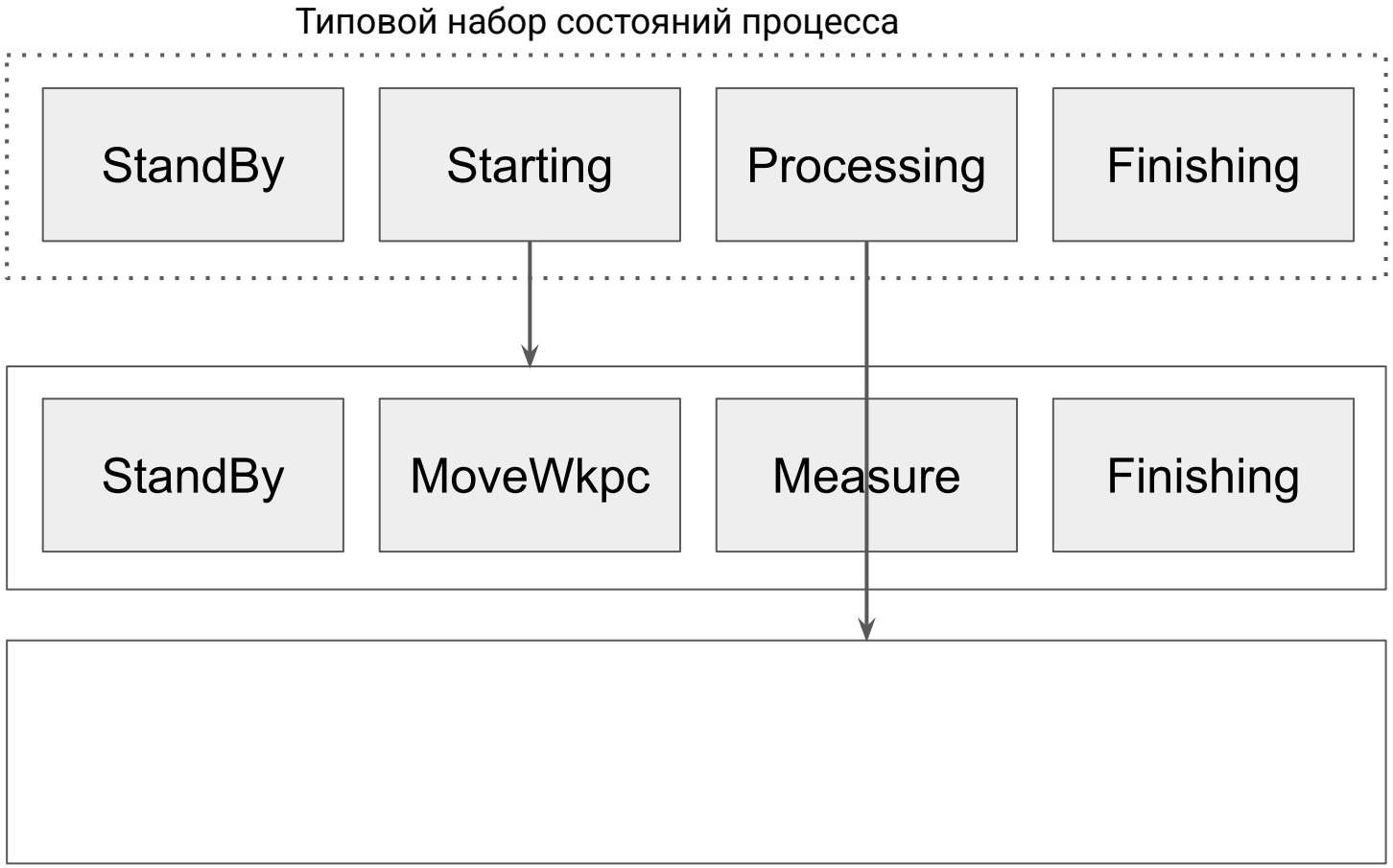
1 Состояния Starting:
- StandBy
- GetTask
- MoveWorkpiece
- MeasureWorkpiece
- FinishPrepareing
- Failure

Состояния можно делить

Дает изоляцию частей КА

Получаем дерево машин состояний





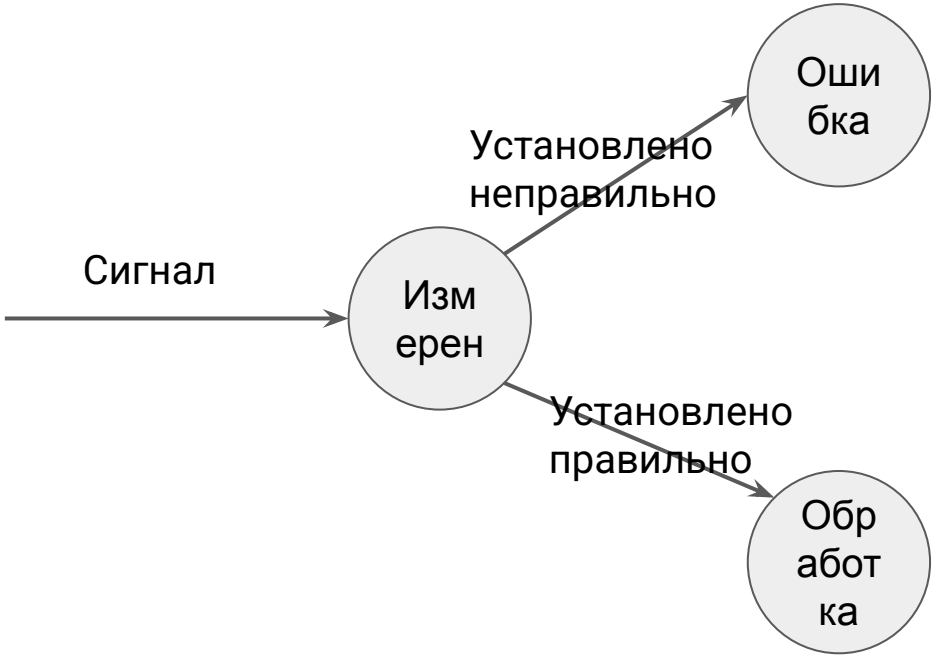
Переходы

(простейший случай)

Словарь вида:

- Ключ (Начальное состояние, сигнал)
- Значение (Конечное состояние, описание)

Текущее состояние	Сигнал	Следующее состояние
Состояние 0	X	Состояние 1
Состояние 1	X	Состояние 2
Состояние 2	X	Состояние 3
Состояние 3	X	Состояние 4
Состояние 4	X	Состояние 5
Состояние 5	X	Состояние 6
Состояние 6	X	Состояние 7
Состояние 7	X	Состояние 8
Состояние 8	X	Состояние 0



Сигналы

Сигналов может быть много.
Каждый сигнал может иметь свой алфавит

Сигнал измерения:
расстояние до заготовки из трех точек
12 mm, 45 mm, 72 mm
Допуски: 10-14 mm; 40-44 mm; 74-79mm
Within, Higher, Lower

	Сигнал 1	**Сигнал 2**	**Сигнал 3**		
Алфавит сигнала:	W, H, L	W, H, L	W, H, L		Всего сигналов: 3*3*3 = 27
Мощность алфавита:	3	3	3		Всего переходов: состояний * сигналов = 8*27 = 216
Текущее состояние				**Следующее состояние**	**Описание**
Состояние 0	W	W	W	Состояние 1	Заготовка встала правильно, переходим в режим обработки
Состояние 0	W	W	H	Состояние 8	Заготовка выставляется по длине, требуется вмешательство оператора
Состояние 0	W	W	L	Состояние 8	- " -
Состояние 0	W	H	W	Состояние 8	- " -
Состояние 0	W	H	H	Состояние 2	- " -
Состояние 0	W	H	L	Состояние 2	- " -
Состояние 0	W	L	W	- " -	- " -
- " -	- " -	- " -	- " -	- " -	- " -

Проблема числа переходов

Очень быстро растет число переходов: $N_{tot} = N_{states} \times N_{s1} \times N_{s2} \times N_{s3} \times \ldots \times N_{sn}$

Выгоднее сделать несколько небольших КА, чем один большой

	Число сигналов						
Мощ.алф.	1	2	3	4	5	6	7
1	1	1	1	1	1	1	1
2	2	4	8	16	32	64	128
3	3	9	27	81	243	729	2187
4	4	16	64	256	1024	4096	16384
5	5	25	125	625	3125	15625	78125
6	6	36	216	1296	7776	46656	279936
7	7	49	343	2401	16807	117649	823543
8	8	64	512	4096	32768	262144	2097152
9	9	81	729	6561	59049	531441	4782969
10	10	100	1000	10000	100000	1000000	10000000

Еще раз

Полный набор возможных переходов: $N_{tot} = N_{states} \times N_{s1} \times N_{s2} \times N_{s3} \times \ldots \times N_{sn}$

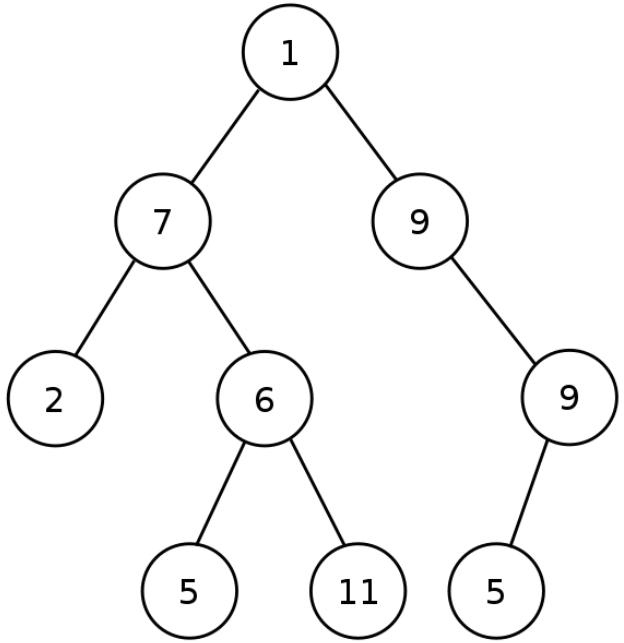
Зачем?

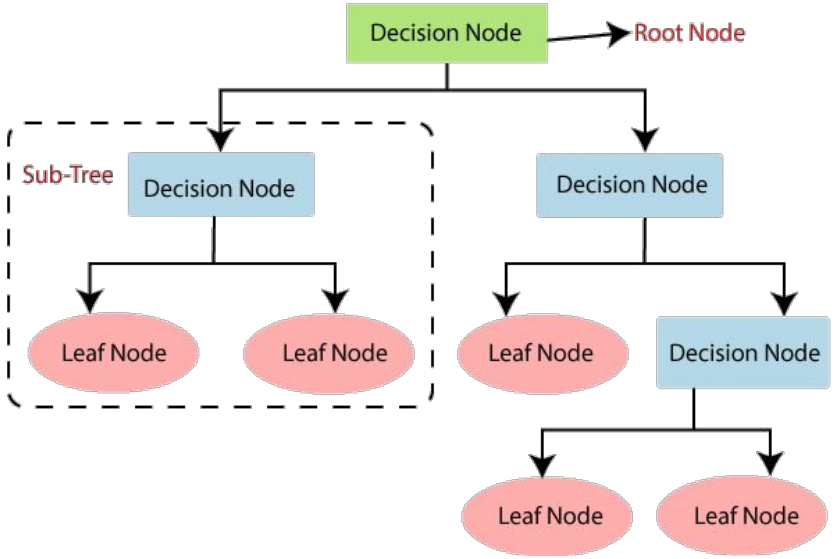
- Позволяет качественно проанализировать всю машину состояний, учесть все без исключения возможности
- Для анализа, т.е. при моделировании всех состояний возникают вопросы бизнеса

Оптимизация машины состояний

Сигналы: способы оптимизации

1. “Облегченная” матрица переходов
2. Композитный сигнал
3. Декомпозиция конечных автоматов
4. Машинное обучение





Облегченный вариант

Содержит только положительные переходы

Сигналы: облегченный вариант vs полный

Облегченный:

- Проще в проектировании и разработке
- Меньше занимает в памяти (несущественно)
- Быстрее (неправда)
- Более слабая проработка логики
- Неконтролируемые сигналы

Полный:

- Сложнее и утомительнее в проектировании
- Больше занимает в памяти (несущественно)
- Не быстрее (неправда)
- Более качественная проработка логики
- Полный контроль над переходами

Срочно позвоните по телефону +7 222 111 0000 и спросите Михаила.
Он вас убедит, что вы не можете видеть это сообщение

Композитный сигнал

Возможен иногда, если есть сценарные сигналы

Пример:

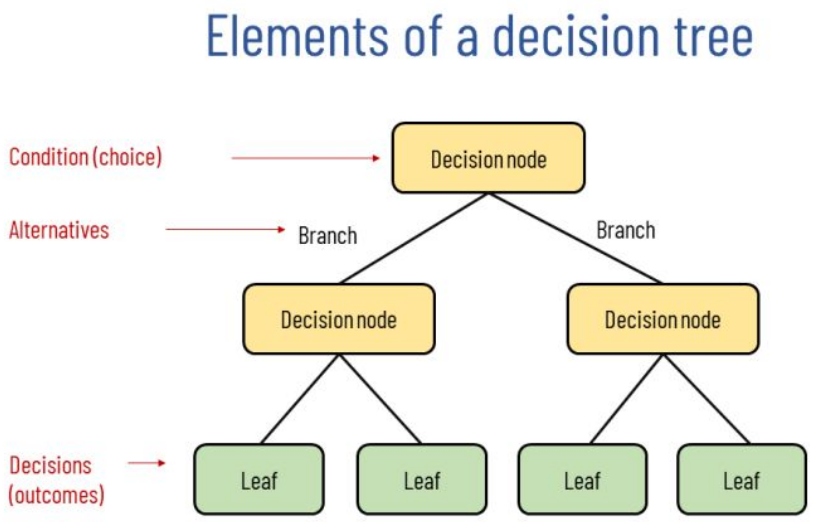
- 1,1,1 - Signal-1
- все остальное - Signal-2

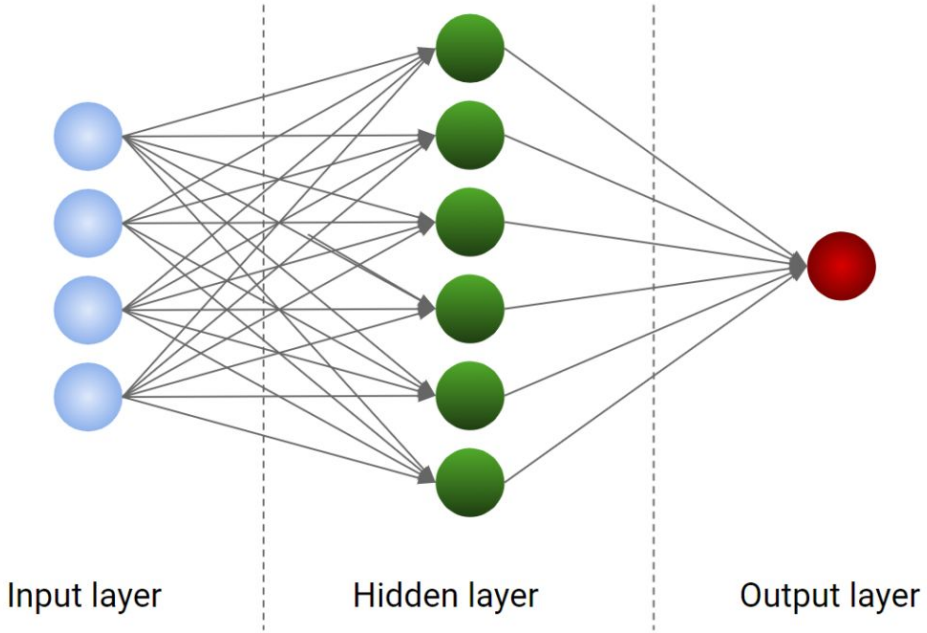
потеря информации о сигналах

Машинное обучение

Варианты:

- Деревья решений
- Нейронная сеть





Машина состояний vs Машинное обучение

Машина состояний:

- Явно прописанная логика, согласованная между всеми сторонами
- Абсолютная предсказуемость
- Быстрее
- Сильное ограничение по числу входных параметров (сигналов)

Дерево решений:

- Квазистохастическое обучение
- Предсказуемость только основных сценариев
- Размер обучающей выборки
- Большое число входных параметров
- Ошибается

Нейронная сеть:

- Квазистохастическое обучение
- Предсказуемость только основных сценариев
- Размер обучающей выборки
- Единственно возможный механизм при больших объемах входных параметров
- Очень большое число входных параметров
- Ошибается

Машина состояний + Машинное обучение

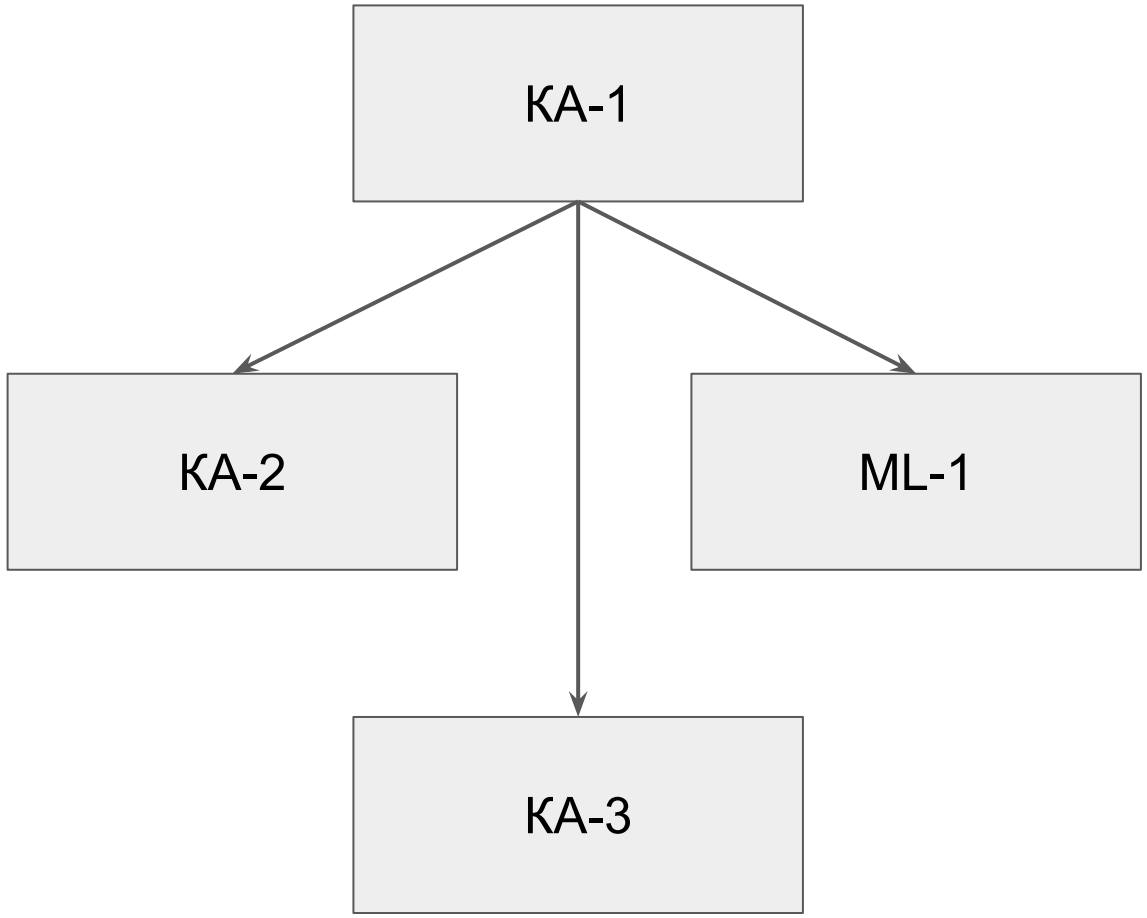
Декомпозиция КА, часть автоматов заменяем на ML

Машинное обучение:

- Обслуживание нечеткой логики с большим числом параметров
    - распознавание композитного сигнала

Машина состояний:

- Обслуживание контрактной логики



Объявления в маркетплейсе

Использование в маркетплейсе

Вопросы проработки:

- Для чего нужны состояния?
- Какие состояния?
- Какие сигналы?
- По каким триггерам выполнять расчет?
- Откуда брать данные для сигналов?
- Хранить ли вычисленные состояния?
- Где хранить?

Рассматриваемый пример нереалистичный. SM больше подходит для более сложных кейсов.
Рассматривается для обсуждения машины состояний.

Использование в маркетплейсе

Вопросы проработки:

- **Для чего нужны состояния?**
- Какие состояния?
- Какие сигналы?
- По каким триггерам выполнять расчет?
- Откуда брать данные для сигналов?
- Хранить ли вычисленные состояния?
- Где хранить?

- Для подсветки объявлений
- Для привлечения пользователей
- Для отображения на UI

Другие варианты:

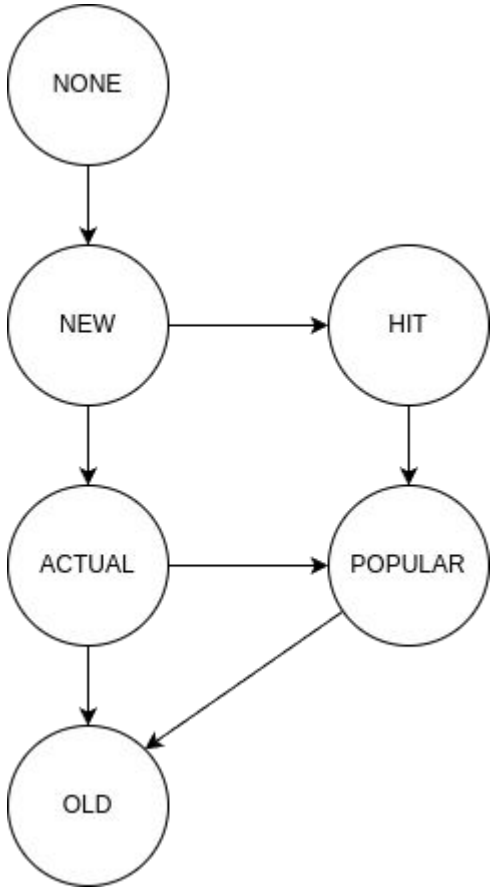
- Для управления другими процессами
- Для изменения алгоритма обработки
- и т.д.

Использование в маркетплейсе

Вопросы проработки:

- Для чего нужны состояния?
- **Какие состояния?**
- Какие сигналы?
- По каким триггерам выполнять расчет?
- Откуда брать данные для сигналов?
- Хранить ли вычисленные состояния?
- Где хранить?

- Черновик
- Новое
- Актуальное
- Старое
- Хит
- Популярное



Использование в маркетплейсе

Вопросы проработки:

- Для чего нужны состояния?
- Какие состояния?
- **Какие сигналы?**
- По каким триггерам выполнять расчет?
- Откуда брать данные для сигналов?
- Хранить ли вычисленные состояния?
- Где хранить?

- Возраст объявления
  - Время создания
  - Время обновления (??)
- Количество просмотров

Использование в маркетплейсе

```
Вопросы проработки:
-   Для чего нужны состояния?
-   Какие состояния?
-   Какие сигналы?
-   По каким триггерам выполнять расчет?
-   Откуда брать данные для сигналов?
-   Хранить ли вычисленные состояния?
-   Где хранить?
```

1. Вычислять при сигналах (read, search, …), сохранять в сущности
2. Вычислять при отдаче клиенту (все методы)
3. Вычислять периодически по времени и сохранять
   a. пониженная нагрузка
   b. примерный результат
   c. независимая логика

Использование в маркетплейсе

Вопросы проработки:

- Для чего нужны состояния?
- Какие состояния?
- Какие сигналы?
- По каким триггерам выполнять расчет?
- **Откуда брать данные для сигналов?**
- Хранить ли вычисленные состояния?
- Где хранить?

Вычислять в логике:

- Нарушение зон ответственности
-

Извлекать из мониторинга:

- Переиспользуем штатную подсистему
- Дешевле в реализации
- Можно вынести в сайдкар или оставить внутри контейнера

Использование в маркетплейсе

Вопросы проработки:

- Для чего нужны состояния?
- Какие состояния?
- Какие сигналы?
- По каким триггерам выполнять расчет?
- Откуда брать данные для сигналов?
- **Хранить ли вычисленные состояния?**
- Где хранить?

Не хранить, вычислять каждый раз

- Повышенная нагрузка на мониторинг
- Для отображения на UI

Хранить:

- Обновления раз в 1-10 минут
- Низкая нагрузка на мониторинг

Использование в маркетплейсе

Вопросы проработки:

- Для чего нужны состояния?
- Какие состояния?
- Какие сигналы?
- По каким триггерам выполнять расчет?
- Откуда брать данные для сигналов?
- Хранить ли вычисленные состояния?
- **Где хранить?**

Внутри объявлений:

- Проще строить бизнес-логику
- Зацепление разных логик

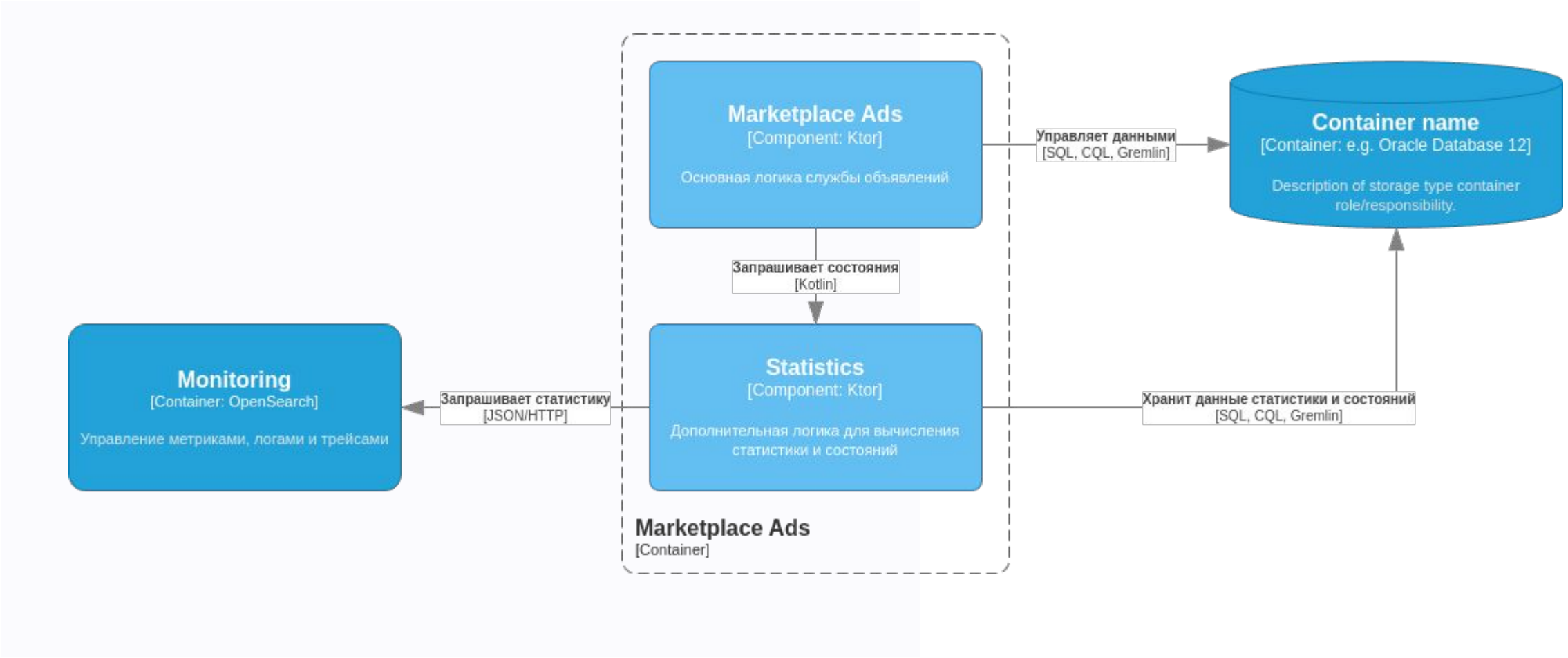
В отдельной таблице:

- Логика не страдает
- Разносим разные сущности

В отдельной БД:

- Свобода выбора типа БД
- Винегрет разные БД усложняет и удорожает проект

Использование в маркетплейсе



LIVE

Рефлексия

Основные тезисы

1.	Машина состояний - абстрактная мат.модель, состоит из состояний, сигналов и переходов
2.	Машины состояний декомпозируются для упрощения проектирования и снижения числа переходов
3.	Машины состояний успешно сочетаются с деревьями решений и нейросетями
4.	Реализация машины состояний выполняется в виде HashMap, что обеспечивает сложность вычислений O(1)
5.	Ключем HashMap служит data class, содержащий начальное состояние и входные сигналы; значением - data class с описанием перехода и следующего состояния

Рефлексия

По какому вопросу захотелось глубже изучить информацию?

Понимаете ли вы, как применять на практике то, что узнали на вебинаре. Если да, то как?

Цели вебинара

Проверка достижения целей

1. Знать общие принципы работы машины состояний
2. Применять их при проектировании логики приложений
3. Реализовывать в своих программах

Список материалов для изучения

- https://kotlinlang.org/
- https://gradle.org/
- https://github.com/otuskotlin/otuskotlin-user
- https://github.com/otuskotlin/202505-ok-marketplace
- https://ru.wikipedia.org/wiki/Конечный_автомат
- https://tproger.ru/translations/finite-state-machines-theory-and-implementation/

Следующий вебинар

21.10.2025

Модуль 8: Развитие бизнес-логики проекта

Авторизация и аутентификация в бэкенде

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары